

B4F Board — System Construction Map

A student guide to the system we are building, why we started where we did, and the exact path we are following until we reach the final product.

Last updated: 2026-10-06 — grounded in the actual current state of the BootCamp (Session 9 is prepared, not yet delivered; Session 8 is the latest actually-taught session).

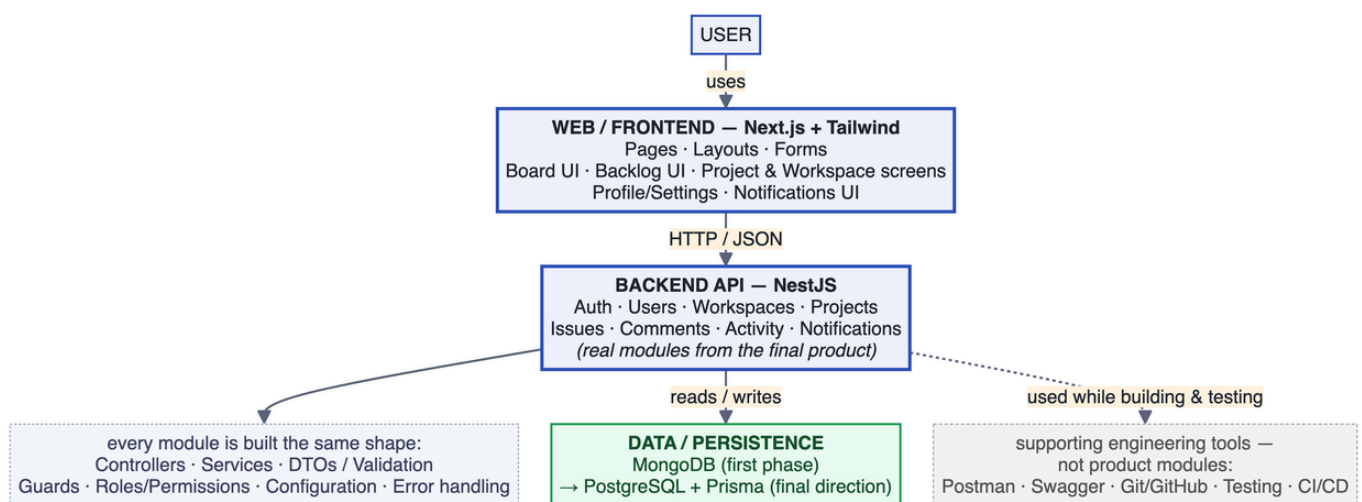
1. What are we actually building?

B4F Board is a real, working project-and-task management system — Jira-style. Not a toy exercise. By the end of this program it will have real users, real data, a real web interface, and a real API enforcing real rules.

This document is a map of that whole system: its pieces, how they connect, which pieces already exist in the classroom repository, and the exact construction order we're following to get from where we are today to the finished product.

2. The whole system map

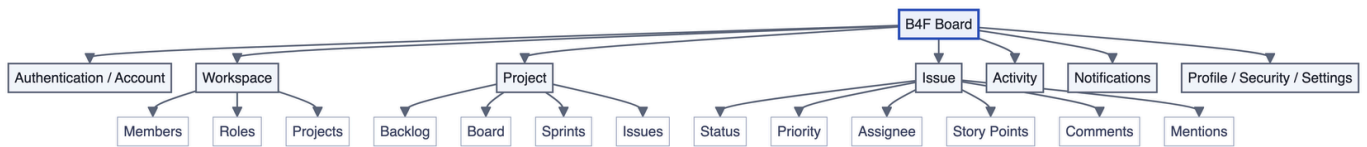
This is the shape of the finished B4F Board. Every page after this one zooms into one part of it.



- **Frontend talks to the backend.** The backend owns the rules — nothing is trusted just because the frontend sent it.
- **Every backend module is built the same way** — a controller, a service, DTOs — which is why adding a new product area (Issues, Comments...) later follows a known, safe pattern.
- **Postman, Swagger, Git, testing, CI/CD** are engineering tools we use *while building* the system. They are not parts of the product itself — that distinction matters throughout this document.

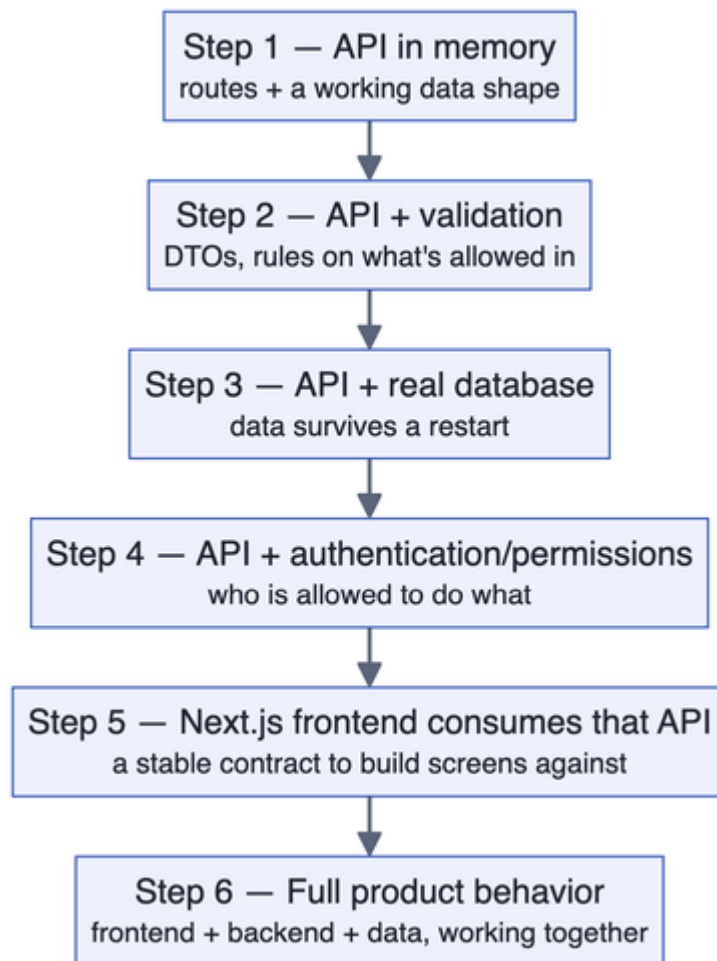
3. What is B4F Board made of?

When we say "a Jira-like app," these are the actual pieces inside it:



This is the **product** shape — not the technology. Every box above eventually becomes real screens, real API routes, and real data. The next pages show exactly how.

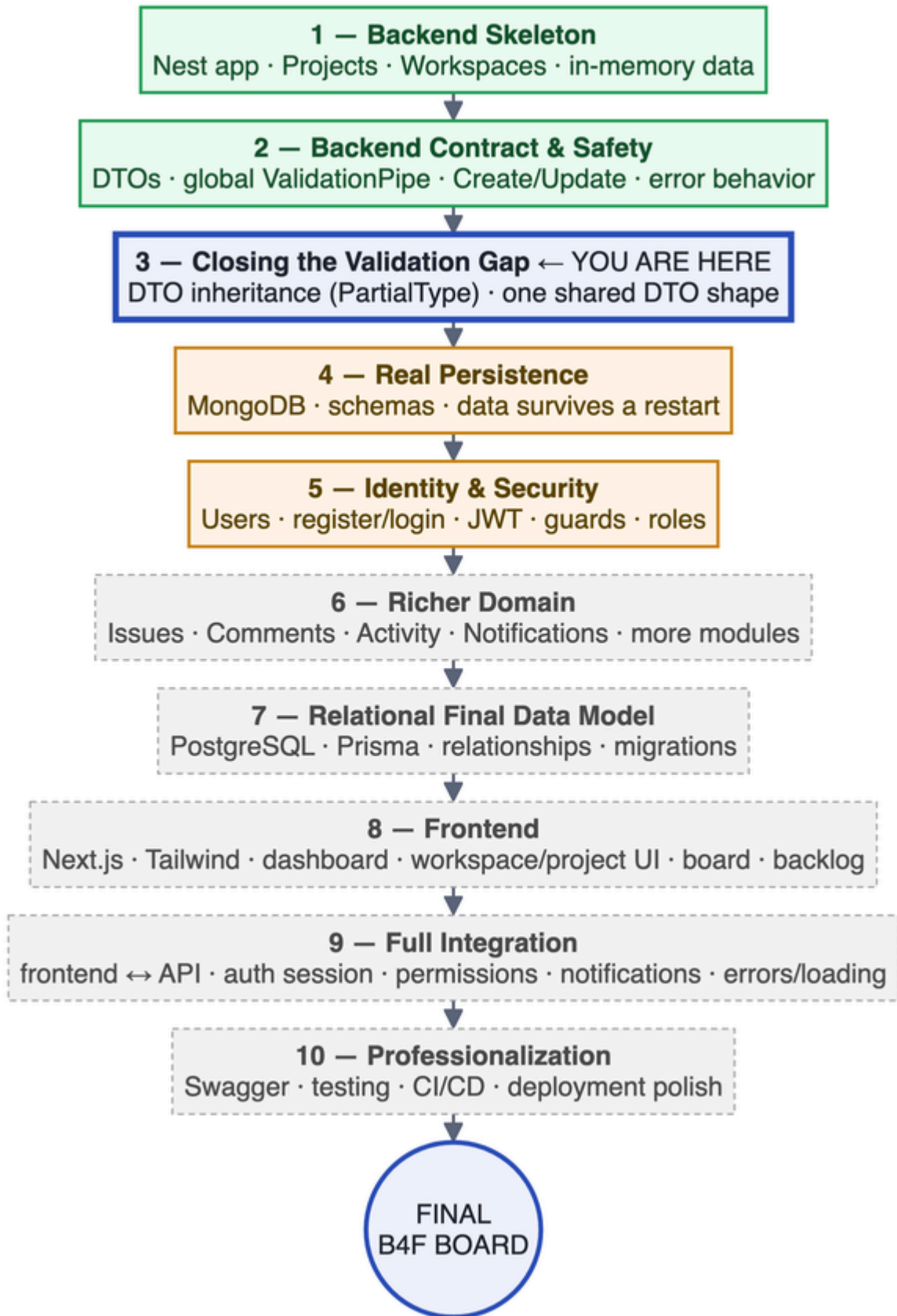
4. Why are we building the backend first?



- **The frontend needs something real to talk to.** A polished frontend built before a real API and data model means faked data or constant rewrites later. The real contract comes first.
- **The backend defines the rules.** *Can this project be created? Can this user update this issue?* Not frontend questions — the backend is the trusted authority that decides.
- **Data relationships come before rich screens.** The final UI shows `Workspace → Projects → Issues → Comments` — we implement how those relate before drawing the screens for them.

Not a claim that backend-first is the only valid professional strategy — for this program and product, it gives a clear, dependency-safe learning order.

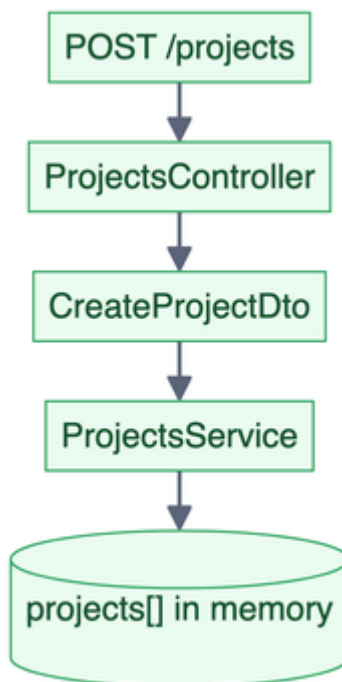
5. Our construction path — you are here



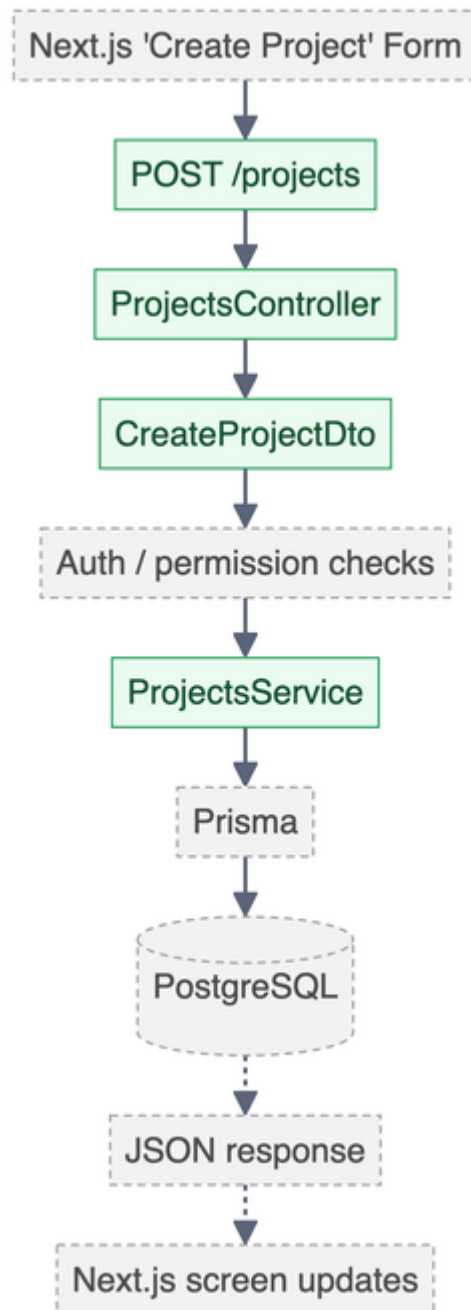
Green = already built and taught. Blue (thick border) = current — being taught right now. Orange = next on the roadmap. Gray (dashed) = later, including the final-stage pieces.

6. What today's NestJS code becomes

Right now you're writing code that looks like this:



That is not a throwaway exercise. The exact same controller, DTO, and service stay in the final system — we attach more infrastructure *around* them. This is what the same feature looks like once the rest of the system is connected:

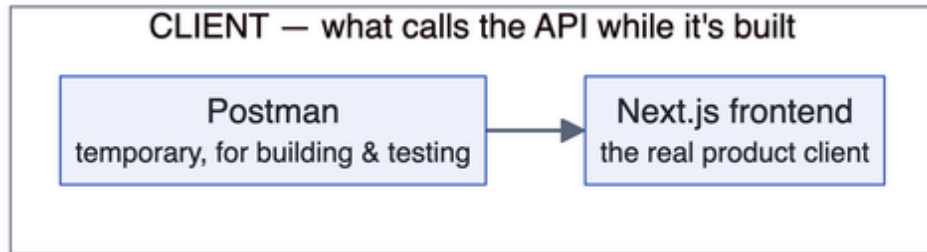
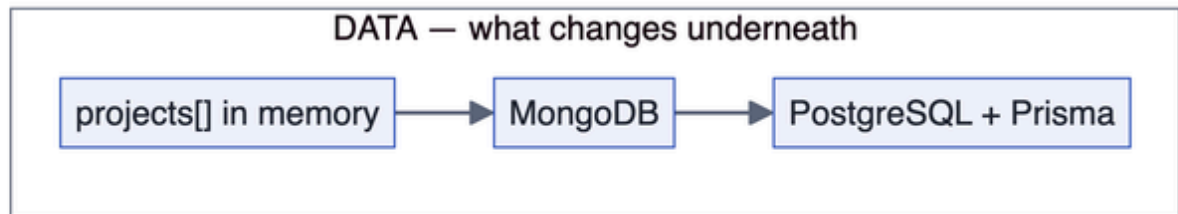


The controller, the DTO, and the service you are writing today do not get replaced. They get connected to more things.

7. Evolution, not replacement

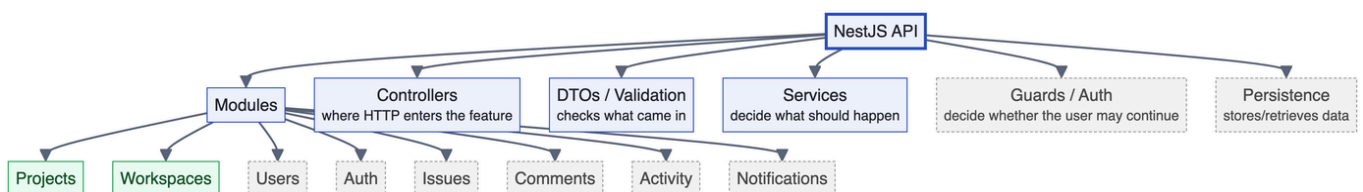
A few honest questions worth answering directly:

Why build in-memory arrays if we're going to use a database? Why build the API before the frontend?
Why MongoDB if PostgreSQL comes later?



Postman is not the product's frontend. It is the temporary client we use to build and test the backend by hand, before a real UI exists to do that job. Today: Postman → Nest API → memory . Later: Next.js → Nest API → PostgreSQL . **The Nest API stays in the middle through all of it** — that's the piece staying stable while everything around it grows up.

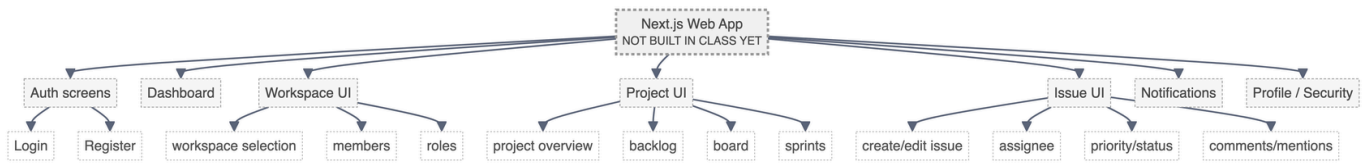
8. Backend map



- **Controller** = where HTTP enters the feature.
- **DTO** = checks what came in.
- **Service** = decides what should happen.
- **Guard** = decides whether the user may continue.
- **Persistence** = stores/retrieves the data.

Projects and Workspaces are real, delivered modules. The rest of the module list is real product scope from the final build — not yet reached in class.

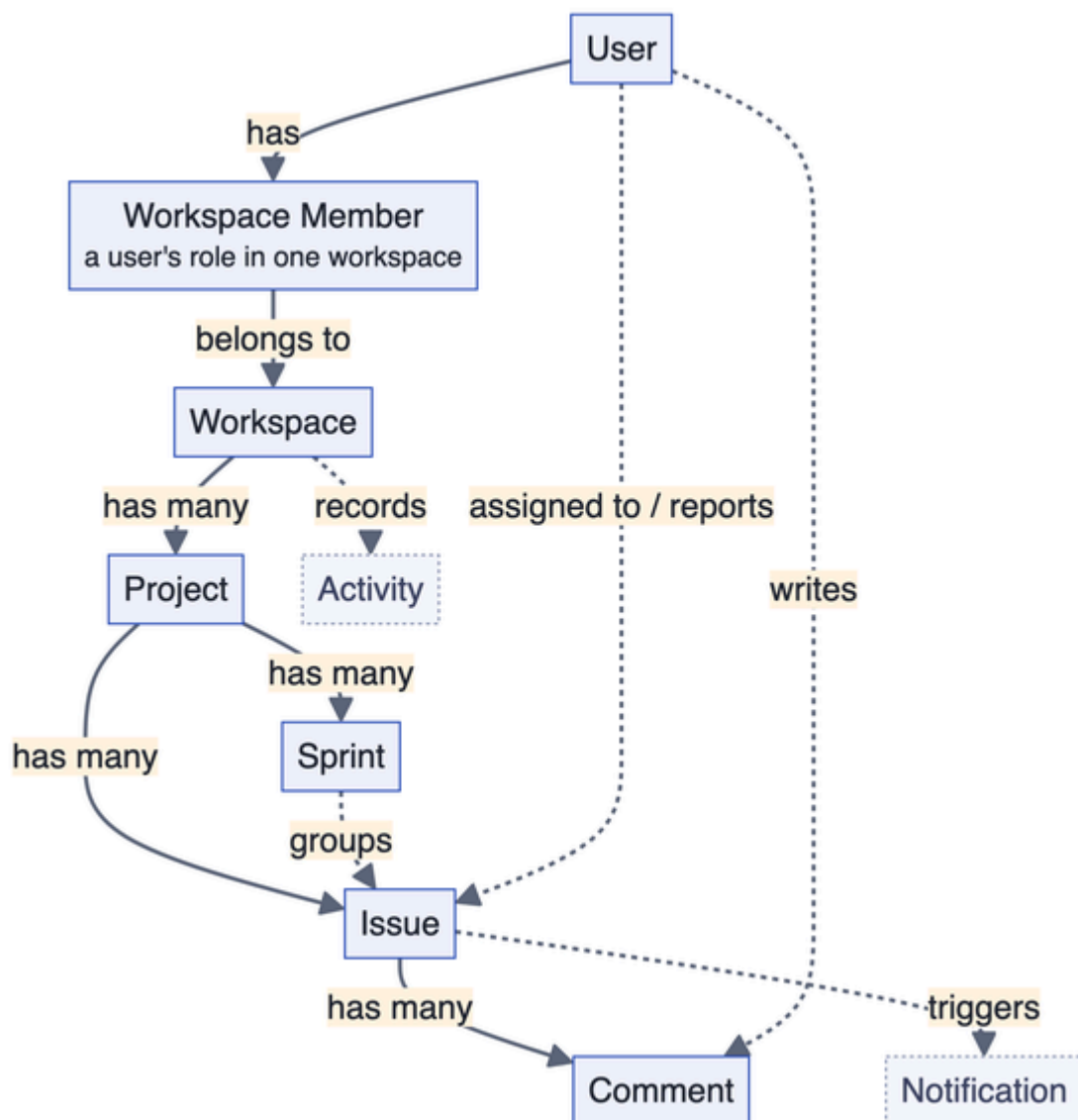
9. Frontend map



The entire Next.js tree above is **not built in class yet** (Block 6 on the roadmap). It is shown here so the shape of the destination is clear — not as a claim about current progress. Frontend isn't "the whole project" any more than backend or database is — the project is all of them, connected.

10. Data relationship map

Conceptual map — not the full database schema. Just enough to see how the real pieces relate:

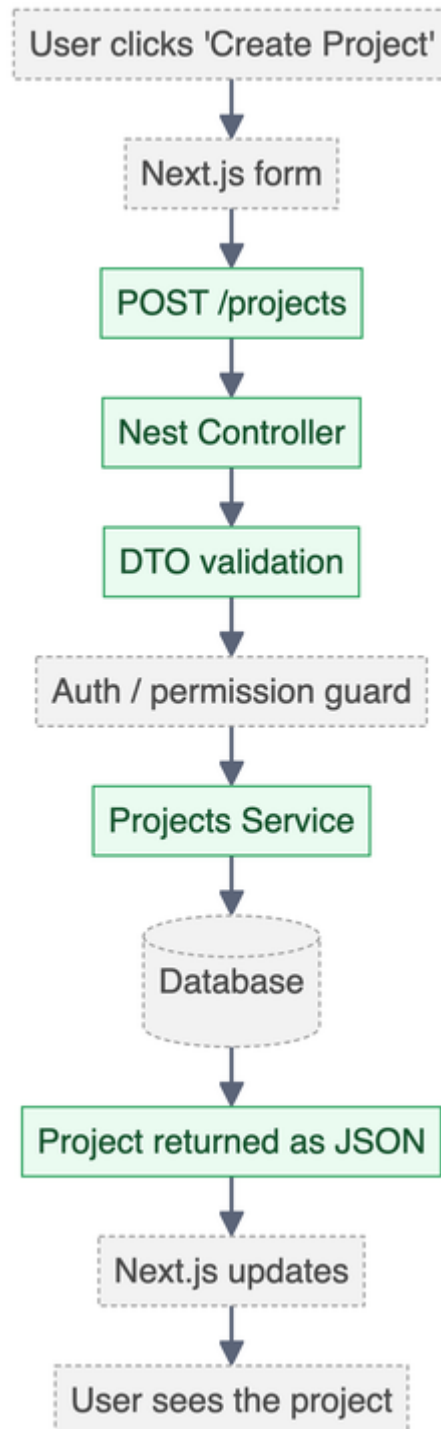


A **User** has a role in a **Workspace** (via **Workspace Member**); a **Workspace** holds **Projects**; a **Project** holds **Issues** (optionally grouped by **Sprint**); an **Issue** holds **Comments**. **Activity** and **Notifications** are cross-cutting — triggered by what happens elsewhere in the system, not features on

their own.

11. One feature through the whole stack

Using **Create Project**, because you already know it:



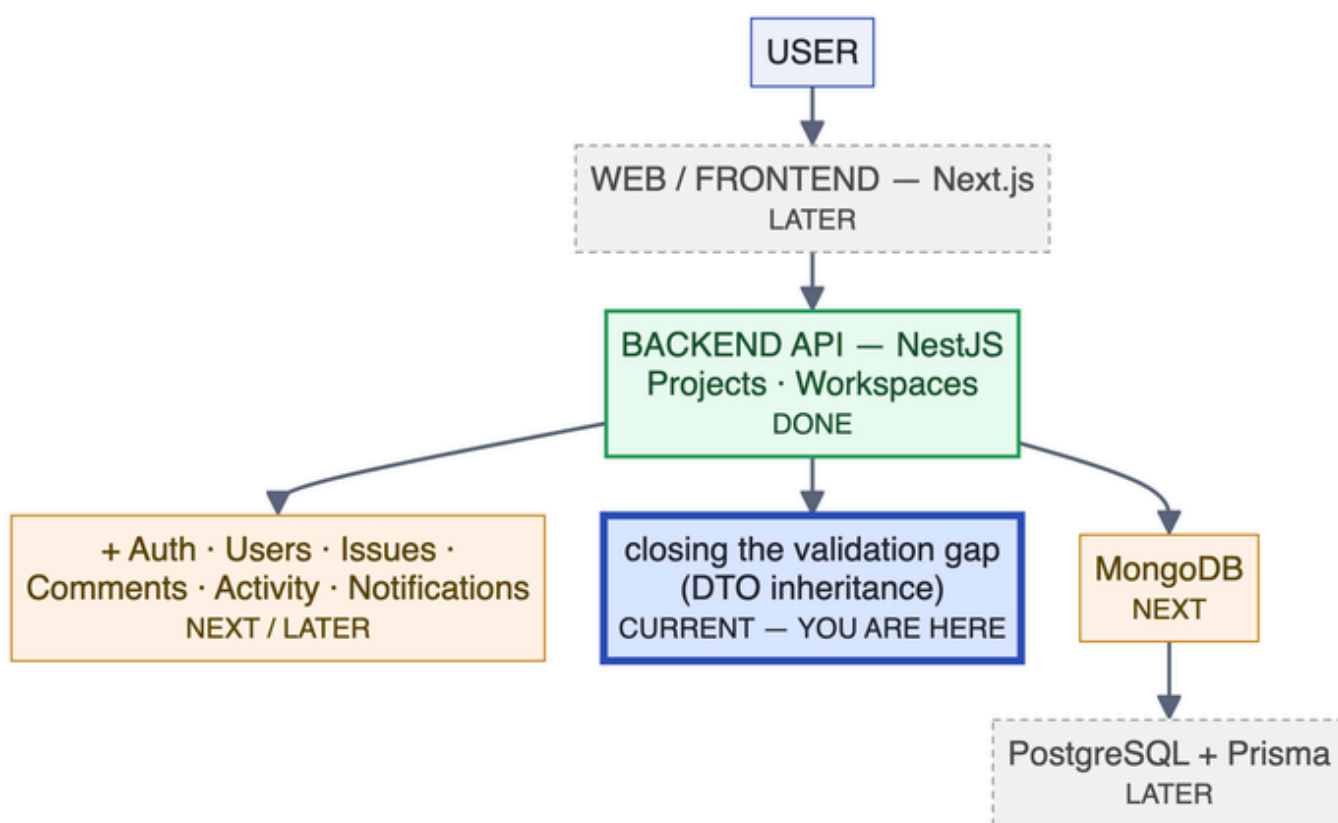
Already built: the controller, the DTO validation, the service, and the JSON response. **Still coming:** the real on-screen form, the auth/permission check, and a real database underneath. This single diagram connects today's class directly to the finished product.

12. Before B4F Board



B4F Hub is history, not the main story here — it taught the ingredients (React state and routing, HTTP, Express, frontend ↔ backend communication) that B4F Board now applies to one real, structured product.

13. Final destination



GREEN = already built and taught · **BLUE (thick border)** = current, being taught right now · **ORANGE** = next on the roadmap · **GRAY (dashed)** = later, including the final-stage pieces.

Every session between now and the Hackathon adds one more real piece to this exact picture. You are not learning isolated topics — you are constructing B4F Board, one solid layer at a time.